

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Trabajo Integrador N°1

Base de datos I

Docente

Samuel Tocaimaza

Estudiantes

Monzon, Martin

Olivero, Christian

Pichulman, Miguel

Rodriguez, Joaquin

23 de octubre de 2025

Índice

1	Etapa 1: Modelado y Constraints	4
1.1	Descripción del dominio elegido.....	4
1.2	Diagrama Entidad–Relación con entidades, atributos y relaciones.	4
1.3	Modelo Relacional en SQL	5
1.4	Capturas de pantalla o resultados en consola.	7
1.4.1	Reflexión sobre lo aprendido.	8
2	Etapa 2: Implementación y Carga Masiva.....	9
2.1	Estrategia de Generación de Datos.....	9
2.1.1	Procedimiento usado.	9
2.2	Carga Masiva.....	10
2.3	Verificaciones de Consistencia	12
2.3.1	Verificar Usuarios con Referencia Inválida:.....	12
2.4	Resultados esperados y observados.....	13
2.5	Evidencia de Interacción con IA	13
3	Etapa 3: Consultas Avanzadas y Reportes	16
3.1	Consultas Avanzadas.....	16
3.1.1	Índices y Medición de Performance	18
3.2	Creación de Vistas.....	19
3.3	Evidencia de Interacción con IA	20
4	Etapa 4: Seguridad e Integridad.....	21
4.1	Creación de Usuario con Privilegios Limitados.....	21
4.2	Vistas para Ocultar Datos Sensibles.....	22
4.2.1	Ejemplo de resultado (antes / después).	23
4.3	Pruebas de Integridad	24
4.3.1	Consulta Parametrizada Segura	25
4.4	Explicación del enfoque de seguridad.....	27
4.5	Evidencia de Interacción con IA	27
5	Etapa 5: Concurrencia y Transacciones.....	29
5.1	Simulación de Deadlock.....	29
5.2	Manejo de Transacciones	30
5.2.1	Procedimiento almacenado con BEGIN/COMMIT/ROLLBACK, manejo de errores y retry ante deadlock si aplica implementado en SQL.	32

5.3	Informe de Observaciones.....	34
5.4	Evidencia de Interacción con IA	35
6	Anexo.....	36

1 Etapa 1: Modelado y Constraints

1.1 Descripción del dominio elegido.

Partiendo del enunciado del CRUD en JAVA el dominio elegido es el de Usuario - CredencialAcceso. Consta de dos Entidades: Usuario y Credencial de Acceso.

Clase Usuario (A):

id (Long, PK): Identificador único de cada usuario.

eliminado (Boolean): Baja lógica, indica si el usuario fue desactivado.

username (String, NOT NULL, UNIQUE, máx. 30): Nombre de usuario, único en el sistema.

email (String, NOT NULL, UNIQUE, máx. 120): Correo electrónico, único y obligatorio.

activo (Boolean, NOT NULL): Estado activo/inactivo de la cuenta.

fechaRegistro (LocalDateTime): Fecha de creación del usuario.

credencial (CredencialAcceso, 1→1): Relación uno a uno con la entidad CredencialAcceso.

Clase CredencialAcceso (B)

id (Long, PK): Identificador único de la credencial.

eliminado (Boolean): Baja lógica de la credencial.

hashPassword (String, NOT NULL, máx. 255): Contraseña cifrada.

salt (String, máx. 64): Valor aleatorio para fortalecer el hash de la contraseña.

ultimoCambio (LocalDateTime): Fecha y hora del último cambio de contraseña.

requiereReset (Boolean, NOT NULL): Indica si el usuario debe cambiar la contraseña en el próximo inicio de sesión.

1.2 Diagrama Entidad–Relación con entidades, atributos y relaciones.

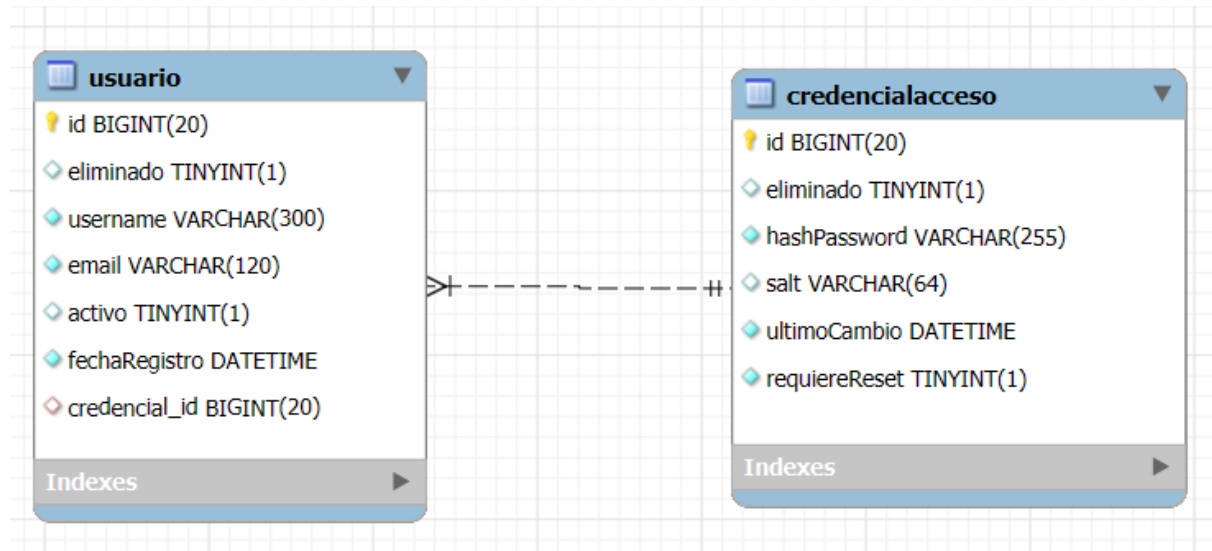
Explicación de cardinalidades y claves.

Usuario – CredencialAcceso

Relación 1→1 (uno a uno)

- Cada Usuario tiene exactamente una CredencialAcceso.
- Cada CredencialAcceso pertenece exactamente a un Usuario.
- Esto asegura que no existan credenciales duplicadas ni usuarios sin credencial.

Captura del DER (imagen insertada).



1.3 Modelo Relacional en SQL

Definición de tablas (con PK, FK, UNIQUE, CHECK y dominios).

Justificación de cada restricción importante.

Se utiliza UNIQUE en username y email para garantizar que no existan duplicados en la identificación de usuarios.

La relación 1→1 se asegura mediante credencial_id con restricción UNIQUE.

El ON DELETE CASCADE evita credenciales huérfanas, preservando la integridad referencial.

Código SQL correspondiente:

```
CREATE TABLE trabajofinal.CredencialAcceso(  
    id BIGINT PRIMARY KEY,  
    eliminado Boolean,  
    hashPassword VARCHAR(255) NOT NULL,  
    salt VARCHAR(64),  
    ultimoCambio DATETIME NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
    requiereReset Boolean NOT NULL  
);  
CREATE TABLE trabajofinal.Usuario (  
    id BIGINT PRIMARY KEY,  
    eliminado BOOLEAN,  
    username VARCHAR(30) NOT NULL UNIQUE,  
    email VARCHAR(120) NOT NULL UNIQUE,
```



```

7 19:14:04 INSERT INTO credencialacceso(id, eliminado, hashPassword, salt, ultimoCambio,requier... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0
8 19:14:12 INSERT INTO usuario (id, eliminado, username, email, activo, fechaRegistro, credencial... 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
9 19:14:48 select * from usuario LIMIT 0, 500 2 row(s) returned

```

- **Inserciones erróneas (que disparan errores distintos).**

```

INSERT INTO usuario (id, eliminado, username, email,
activo, fechaRegistro, credencial_id )
VALUES
(4,false, 'Usuario3',null, '', '2025-10-15',1)
;

```

Error Code: 1048. Column 'email' cannot be null

```

INSERT INTO usuario (id, eliminado, username, email,
activo, fechaRegistro, credencial_id )
VALUES
(4,false, 'Usuario2','Usuario2@gmail.com', true, '2025-10-
15',2)
;

```

Error Code: 1062. Duplicate entry 'Usuario2' for key 'username'

1.4 Capturas de pantalla o resultados en consola.

Evidencia de Interacción con IA

Breve explicación de cómo se usó IA (por ejemplo: revisión de modelo, sugerencias de constraints, etc.).

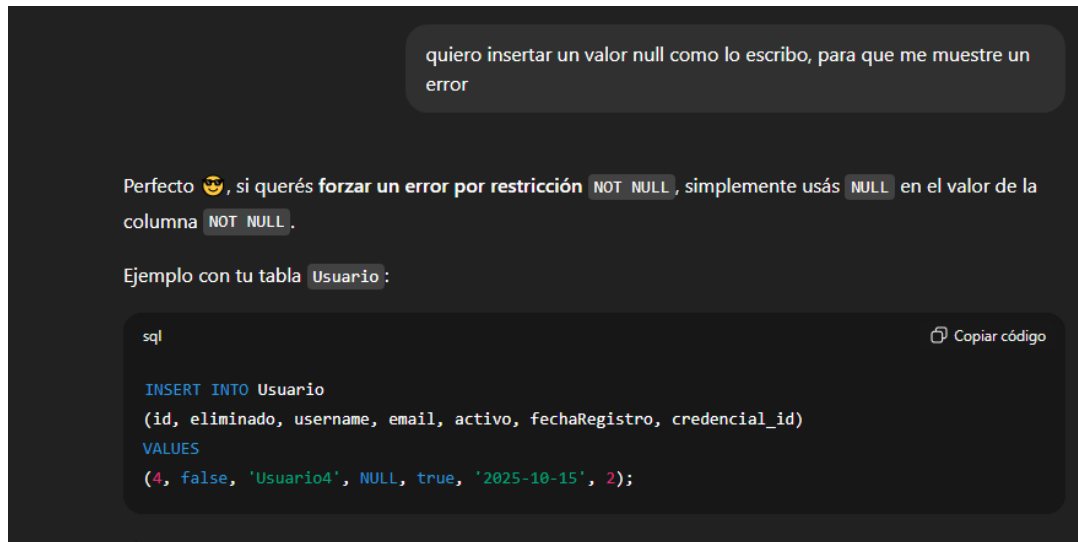
- Se consultó la IA para saber como cargar un tipo de dato “fecha” en la creacion de la tabla

tengo que agregar esta sentencia
fechaRegistro java.time.LocalDateTime

Ah, perfecto 😊. En SQL/MariaDB no existe `java.time.LocalDateTime`; hay que usar **tipos de fecha/hora** que SQL entiende:

- **DATETIME** → almacena fecha y hora (`YYYY-MM-DD HH:MM:SS`)
- **TIMESTAMP** → similar a **DATETIME**, pero puede actualizarse automáticamente con `CURRENT_TIMESTAMP`.

-Consultamos como insertar un valor null en un insert de una tabla.



1.4.1 Reflexión sobre lo aprendido.

A través de este trabajo comprendimos cómo las decisiones de modelado conceptual se reflejan directamente en el modelo físico.

Aplicar restricciones y validar errores nos ayudó a entender cómo mantener la integridad y consistencia de los datos.

También observamos que una buena definición de claves evita redundancias y facilita la administración de relaciones $1 \rightarrow 1$.

2 Etapa 2: Implementación y Carga Masiva

2.1 Estrategia de Generación de Datos

Explicación del enfoque (SQL puro, funciones, loops, etc.).

Para la carga masiva de datos se optó por un Procedimiento Almacenado en SQL puro, ya que permite ejecutar una lógica compleja de generación de datos de forma masiva y eficiente directamente en el motor de la base de datos. La estrategia se centra en un bucle WHILE que repite el proceso de inserción para alcanzar el volumen de registros deseado. En cada iteración, para simular la realidad, se generan nombres a partir de listas predefinidas utilizando las funciones ELT() y RAND() (para seleccionar aleatoriamente nombres y apellidos comunes sin acentos). A continuación, se construyen dinámicamente formatos de username y email que imitan a los de un sistema real, usando CONCAT() y SUBSTRING() (para unir la inicial del nombre, el apellido, un contador y un dominio de correo). Finalmente, para garantizar la integridad referencial y la correcta relación 1 a 1, se inserta primero el registro en la tabla CredencialAcceso y, acto seguido, se utiliza ese mismo ID para crear el registro correspondiente en Usuario.

2.1.1 Procedimiento usado.

```
DELIMITER $$
```

```
USE trabajofinal$$
```

```
CREATE PROCEDURE CargarUsuariosR(IN cantidad_usuarios INT)
BEGIN
    DECLARE i INT DEFAULT 1;
    DECLARE v_nombre VARCHAR(50);
    DECLARE v_apellido VARCHAR(50);
    DECLARE v_username VARCHAR(30);
    DECLARE v_email VARCHAR(120);
    DECLARE v_dominio VARCHAR(50);

    WHILE i <= cantidad_usuarios DO
        SET v_nombre = ELT(FLOOR(1 + RAND() * 10), 'juan',
'carlos', 'sofia', 'maria', 'martin', 'lucia', 'diego', 'ana',
'pedro', 'laura');
        SET v_apellido = ELT(FLOOR(1 + RAND() * 10),
'gonzalez', 'rodriguez', 'gomez', 'fernandez', 'lopez',
'diaz', 'martinez', 'perez', 'romero', 'sanchez');
        SET v_dominio = ELT(FLOOR(1 + RAND() * 4),
'gmail.com', 'hotmail.com', 'outlook.com', 'empresa.com');
    
```

```

        SET v_username = CONCAT(SUBSTRING(v_nombre, 1, 1),
v_apellido, i);
        SET v_email = CONCAT(v_username, '@', v_dominio);

        INSERT INTO CredencialAcceso (id, eliminado,
hashPassword, salt, requiereReset)
        VALUES (i, FALSE, MD5(CONCAT('pass', i)), UUID(),
(RAND() > 0.95));

        INSERT INTO Usuario (id, eliminado, username, email,
activo, credencial_id)
        VALUES (i, FALSE, v_username, v_email, (RAND() > 0.1),
i);

        SET i = i + 1;
    END WHILE;
END$$

DELIMITER ;

```

2.2 Carga Masiva

Evidencia de la cantidad de registros generados (10.000–200.000).

Fragmento de código SQL usado.
CALL CargarUsuariosR(10000);

#	TIME	PROCESO	MESSAGE	USER/DB/T/RESULT
3	20:36:52	USE trabajofinal	0 row(s) affected	0.000 sec
4	20:36:52	SET FOREIGN_KEY_CHECKS = 0	0 row(s) affected	0.000 sec
5	20:36:52	TRUNCATE TABLE Usuario	0 row(s) affected	0.093 sec
6	20:36:52	TRUNCATE TABLE CredencialAcceso	0 row(s) affected	0.032 sec
7	20:36:52	SET FOREIGN_KEY_CHECKS = 1	0 row(s) affected	0.000 sec
8	20:36:58	CALL CargarUsuariosR(10000)	1 row(s) affected	36.578 sec

Query 1 x

Limit to 1000 rows

```
1 • SELECT COUNT(*) AS 'Total de Usuarios Creados' FROM trabajofinal.Usuario;
```

Result Grid

Total de Usuarios Creados
10000

Filter Rows: | Export: | Wrap Cell Content: |

Result Grid

Form Editor

Field Types

Query Stats

Result 1 x

Read Only

Query 1 x

Limit to 1000 rows

```
1 SELECT
2     id,
3     username,
4     email,
5     activo,
6     fechaRegistro
7 FROM
8     trabajofinal.Usuario
9 LIMIT 15;
```

Result Grid

	id	username	email	activo	fechaRegistro
1	ldiaz1	ldiaz1@hotmail.com		1	2025-10-16 20:36:58
2	llopez2	llopez2@hotmail.com		1	2025-10-16 20:36:58
3	mrodriguez3	mrodriguez3@hotmail.com		1	2025-10-16 20:36:58
4	mmartinez4	mmartinez4@outlook.com		1	2025-10-16 20:36:58
5	cperez5	cperez5@hotmail.com		1	2025-10-16 20:36:58
6	agonzalez6	agonzalez6@gmail.com		1	2025-10-16 20:36:58
7	sperez7	sperez7@gmail.com		1	2025-10-16 20:36:58
8	msanchez8	msanchez8@gmail.com		1	2025-10-16 20:36:58
9	dgonzalez9	dgonzalez9@hotmail.com		1	2025-10-16 20:36:58
10	dmartinez10	dmartinez10@gmail.com		1	2025-10-16 20:36:58
11	lmartinez11	lmartinez11@gmail.com		1	2025-10-16 20:36:58
12	aromero12	aromero12@empresa.com		1	2025-10-16 20:36:58
13	lfernandez13	lfernandez13@gmail.com		1	2025-10-16 20:36:58
14	smartinez14	smartinez14@gmail.com		1	2025-10-16 20:36:58
15	mlopez15	mlopez15@empresa.com		1	2025-10-16 20:36:58
*	NULL	NULL	NULL	NULL	NULL

Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

Result Grid

Form Editor

Field Types

Query Stats

se realizó la carga de 10 mil datos de usuarios (fiel a la realidad) en nuestra base datos gracias al procedimiento creado y llamarlo CALL CargarUsuariosR(10000); para proceder con la carga , luego se verifico que la carga fuera un exito con el comando SELECT COUNT(*) AS 'Total de Usuarios Creados' FROM trabajofinal.Usuario; y por último se verificaron los datos usando :

```
SELECT
    id,
    username,
    email,
    activo,
    fechaRegistro
FROM
    trabajofinal.Usuario
LIMIT 15;
```

2.3 Verificaciones de Consistencia

Para validar la integridad de la base de datos tras la carga masiva, se ejecutaron tres consultas de consistencia. Primero, se verificó la integridad referencial para asegurar que no existieran registros en la tabla Usuario cuya clave foránea (credencial_id) apuntara a una credencial inexistente. Para ello, se utilizó un LEFT JOIN desde Usuario hacia CredencialAcceso, buscando los casos sin correspondencia; el resultado, tanto esperado como observado, fue de 0 registros, confirmando que cada usuario tiene una referencia válida. A continuación, se realizó la prueba inversa para confirmar que no hubiera credenciales no asignadas a ningún usuario, empleando un LEFT JOIN desde CredencialAcceso hacia Usuario, cuyo resultado también fue 0. Finalmente, para validar la cardinalidad 1 a 1, se realizó una consulta que agrupa los registros por credencial_id y filtra aquellos con un conteo mayor a uno (HAVING COUNT(*) > 1). El resultado observado fue un conjunto vacío, tal como se esperaba, lo que demuestra de manera concluyente que la restricción UNIQUE es efectiva y la relación uno a uno se mantiene íntegra en los 10,000 registros.

Ejemplos de consultas para comprobar integridad referencial y cardinalidades.

2.3.1 Verificar Usuarios con Referencia Inválida:

```
SELECT
    COUNT(*) AS 'Usuarios_con_referencia_invalida'
FROM
    trabajofinal.Usuario u
LEFT JOIN
    trabajofinal.CredencialAcceso ca ON u.credencial_id =
    ca.id
WHERE
    ca.id IS NULL;
```

Verificar Credenciales No Asignadas:

```
SELECT
```

```

COUNT(*) AS 'Credenciales_no_asignadas'
FROM
    trabajofinal.CredencialAcceso ca
LEFT JOIN
    trabajofinal.Usuario u ON ca.id = u.credencial_id
WHERE
    u.id IS NULL;

```

Validar Cardinalidad 1 a 1:

```

SELECT
    credencial_id,
    COUNT(*) AS cantidad_de_asignaciones
FROM
    trabajofinal.Usuario
GROUP BY
    credencial_id
HAVING
    COUNT(*) > 1;

```

2.4 Resultados esperados y observados

La ejecución de las consultas de verificación arrojó resultados que coincidieron exactamente con los valores esperados, confirmando la robustez de la base de datos y de la carga de datos.

La primera consulta, fue diseñada para detectar usuarios con referencias inválidas, devolvió 0, lo que indica que no existen inconsistencias en la clave foránea. La segunda consulta para encontrar credenciales no asignadas a ningún usuario también obtuvo de resultado 0 validando que no hay registros aislados.

Por último, se validó la cardinalidad 1:1 con una consulta diseñada para detectar si alguna credencial fue asignada a múltiples usuarios. El resultado fue un conjunto de datos vacío, lo que demuestra de forma concluyente que la restricción UNIQUE sobre la columna credencial_id es efectiva. Esta prueba confirma que la regla de negocio principal del modelo se mantuvo íntegra y sin violaciones tras la carga masiva de 10.000 registros.

2.5 Evidencia de Interacción con IA

Qué sugerencia aportó la IA para mejorar la carga o la verificación.

Durante el proceso de carga masiva de datos se solicitó la asistencia de Gemini para resolver dos desafíos principales.

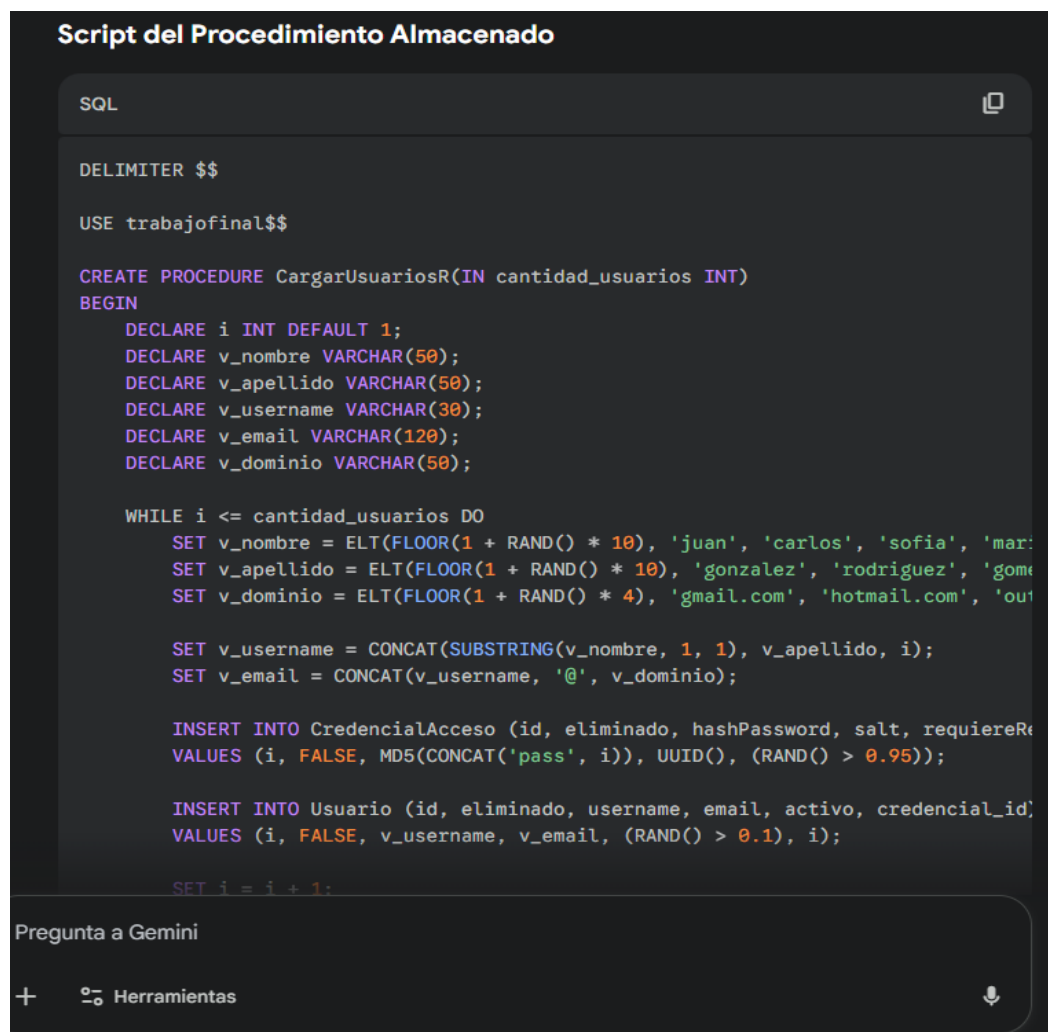
El primero surgió a partir de un script inicial diseñado para generar 10.000 registros aleatorios, el cual producía datos poco realistas al basarse únicamente en funciones simples como RAND() y CONCAT().

Con el objetivo de mejorar la calidad de los datos fuera lo más realista posible, se pidió a Gemini un procedimiento más avanzado, que utilizaba funciones como ELT(), RAND() y CONCAT() de forma estructurada, generando información más coherente.

Durante la ejecución del nuevo código aparecieron los errores Error 2013 (timeout) y Error 1062 (duplicate entry).

El primero se debía a que el script superaba el tiempo máximo de conexión permitido, por lo que se solucionó aumentando el valor del timeout en MySQL Workbench.

Al volver a ejecutar el procedimiento, algunos registros cargados parcialmente durante la interrupción anterior provocaron conflictos por duplicidad, problema que se resolvió mediante un script de limpieza con TRUNCATE TABLE proporcionado por Gemini.



```
SQL

DELIMITER $$

USE trabajofinal$$

CREATE PROCEDURE CargarUsuariosR(IN cantidad_usuarios INT)
BEGIN
    DECLARE i INT DEFAULT 1;
    DECLARE v_nombre VARCHAR(50);
    DECLARE v_apellido VARCHAR(50);
    DECLARE v_username VARCHAR(30);
    DECLARE v_email VARCHAR(120);
    DECLARE v_dominio VARCHAR(50);

    WHILE i <= cantidad_usuarios DO
        SET v_nombre = ELT(FLOOR(1 + RAND() * 10), 'juan', 'carlos', 'sofia', 'maria');
        SET v_apellido = ELT(FLOOR(1 + RAND() * 10), 'gonzalez', 'rodriguez', 'gomez');
        SET v_dominio = ELT(FLOOR(1 + RAND() * 4), 'gmail.com', 'hotmail.com', 'outlook.com');

        SET v_username = CONCAT(SUBSTRING(v_nombre, 1, 1), v_apellido, i);
        SET v_email = CONCAT(v_username, '@', v_dominio);

        INSERT INTO CredencialAcceso (id, eliminado, hashPassword, salt, requiereRe
VALUES (i, FALSE, MD5(CONCAT('pass', i)), UUID(), (RAND() > 0.95));

        INSERT INTO Usuario (id, eliminado, username, email, activo, credencial_id)
VALUES (i, FALSE, v_username, v_email, (RAND() > 0.1), i);

        SET i = i + 1;
    END WHILE;
END
```

Pregunta a Gemini

+ Herramientas

¡Claro! Aquí tienes el script.

Este código se utiliza para vaciar por completo las tablas `Usuario` y `CredencialAcceso` de forma segura, reiniciándolas para una nueva carga de datos. Es útil para evitar errores de duplicidad cuando un proceso de inserción masiva se interrumpe a la mitad.

Script de Limpieza de Tablas

SQL



```
-- 1. Se selecciona la base de datos para asegurar que los comandos se apliquen en
USE trabajofinal;

-- 2. Se desactivan temporalmente las restricciones de clave foránea para permitir
SET FOREIGN_KEY_CHECKS = 0;

-- 3. Se vacían las tablas. TRUNCATE es más eficiente que DELETE para borrar todos
TRUNCATE TABLE Usuario;
TRUNCATE TABLE CredencialAcceso;

-- 4. Se vuelven a activar las restricciones, un paso crucial para mantener la inte
SET FOREIGN_KEY_CHECKS = 1;
```

3 Etapa 3: Consultas Avanzadas y Reportes

3.1 Consultas Avanzadas

- **Consulta 1 (JOIN) → utilidad y resultado.**

```
SELECT u.username, u.email, c.requiereReset,  
c.ultimoCambio  
FROM usuario u  
INNER JOIN credencialacceso c  
ON u.credencial_id = c.id  
WHERE c.requiereReset = TRUE  
;
```

	username	email	requiereReset	ultimoCambio
►	dsanchez8	dsanchez8@empresa.com	1	2025-10-16 21:14:15
	dgomez15	dgomez15@hotmail.com	1	2025-10-16 21:14:15
	ddiaz23	ddiaz23@empresa.com	1	2025-10-16 21:14:15
	adiaz72	adiaz72@outlook.com	1	2025-10-16 21:14:15
	cgonzalez78	cgonzalez78@empresa.com	1	2025-10-16 21:14:15
	pmartinez86	pmartinez86@empresa.com	1	2025-10-16 21:14:15
	cfernandez101	cfernandez101@gmail.com	1	2025-10-16 21:14:15
	jgomez106	jgomez106@empresa.com	1	2025-10-16 21:14:15
	mdiaz122	mdiaz122@outlook.com	1	2025-10-16 21:14:15
	sperez141	sperez141@gmail.com	1	2025-10-16 21:14:15
	aperez153	aperez153@outlook.com	1	2025-10-16 21:14:15
	mgonzalez155	mgonzalez155@gmail.com	1	2025-10-16 21:14:15
	cromero160	cromero160@hotmail.com	1	2025-10-16 21:14:15
	dfernandez236	dfernandez236@gmail.com	1	2025-10-16 21:14:15

La finalidad de esta consulta es ver que usuarios necesitan resetear su contraseña

- **Consulta 2 (JOIN) → utilidad y resultado.**

```
SELECT u.username, u.email, c.ultimoCambio  
FROM usuario u  
INNER JOIN credencialacceso c  
ON u.credencial_id = c.id  
WHERE NOT u.activo  
;
```

	username	email	ultimoCambio
►	ddiaz23	ddiaz23@empresa.com	2025-10-16 21:14:15
	lmartinez41	lmartinez41@hotmail.com	2025-10-16 21:14:15
	dgonzalez48	dgonzalez48@hotmail.com	2025-10-16 21:14:15
	jmartinez59	jmartinez59@empresa.com	2025-10-16 21:14:15
	drodriguez66	drodriguez66@gmail.com	2025-10-16 21:14:15
	sdiaz95	sdiaz95@hotmail.com	2025-10-16 21:14:15
	csanchez105	csanchez105@outlook.com	2025-10-16 21:14:15
	lgonzalez133	lgonzalez133@hotmail.com	2025-10-16 21:14:15
	llopez134	llopez134@outlook.com	2025-10-16 21:14:15
	lmartinez162	lmartinez162@outlook.com	2025-10-16 21:14:15
	pmartinez169	pmartinez169@empresa.com	2025-10-16 21:14:15

Mostrar usuarios inactivos junto con la fecha de su último cambio de clave, con esta consulta podemos verificar que ningún usuario dado de baja siga operando de manera no correspondida

- **Consulta 3 (GROUP BY / HAVING).**

```
SELECT activo, COUNT(*) AS cantidad
FROM usuario
GROUP BY activo
;
```

	activo	cantidad
►	0	1039
	1	8961

Esta consulta cuenta con los usuarios activos y no activos

- **Consulta 4 (Subconsulta).**

```
SELECT u.username , u.email , c.ultimoCambio
FROM usuario u
INNER JOIN credencialacceso c
ON u.credencial_id=c.id
WHERE c.ultimoCambio = (
                        SELECT MAX(ultimoCambio)
                        FROM CredencialAcceso
                        )
;
```

	username	email	ultimoCambio
►	mgomez9565	mgomez9565@gmail.com	2025-10-16 21:14:36
	mmartinez9566	mmartinez9566@gmail.com	2025-10-16 21:14:36
	arodriguez9567	arodriguez9567@outlook.com	2025-10-16 21:14:36
	lrodriguez9568	lrodriguez9568@empresa.com	2025-10-16 21:14:36
	lperez9569	lperez9569@empresa.com	2025-10-16 21:14:36
	sdiaz9570	sdiaz9570@empresa.com	2025-10-16 21:14:36
	mromero9571	mromero9571@empresa.com	2025-10-16 21:14:36
	lsanchez9572	lsanchez9572@gmail.com	2025-10-16 21:14:36
	msanchez9573	msanchez9573@outlook.com	2025-10-16 21:14:36
	jfernandez9574	jfernandez9574@hotmail.com	2025-10-16 21:14:36
	lgomez9575	lgomez9575@hotmail.com	2025-10-16 21:14:36
	crodriguez9576	crodriguez9576@empresa.com	2025-10-16 21:14:36

Esta consulta devuelve los usuarios que hicieron un cambio de clave más reciente.

3.1.1 Índices y Medición de Performance

Creación de índice(s) usados.

Comparación de tiempos con y sin índice.

```
SELECT SQL_NO_CACHE *
FROM CredencialAcceso
WHERE requiereReset = TRUE
ORDER BY ultimoCambio DESC
;
```

Tiempo de ejecucion 0.046sec

Creamos el indice :

```
CREATE INDEX idx_credencial_reset
ON CredencialAcceso(requiereReset);
```

Volvemos a ejecutar:

```
SELECT SQL_NO_CACHE *
FROM CredencialAcceso
WHERE requiereReset = TRUE
ORDER BY ultimoCambio DESC
;
```

Tiempo de ejecucion 0.015 seg

Conclusión breve sobre impacto.

El índice idx_credencial_reset permite filtrar de manera más eficiente las filas que requieren restablecimiento de contraseña, evitando un escaneo completo de la tabla y mejorando el tiempo de respuesta de la consulta.

3.2 Creación de Vistas

Código SQL de la vista.

```
CREATE VIEW v_usuarios_activos AS
SELECT u.username, u.email, u.activo, c.requiereReset, c.ultimoCambio
FROM usuario u
INNER JOIN credencialacceso c
on c.id = u.credencial_id
WHERE u.activo
;
```

	username	email	activo	requiereReset	ultimoCambio
►	lgonzalez1	lgonzalez1@hotmail.com	1	0	2025-10-16 21:14:15
	pgonzalez2	pgonzalez2@empresa.com	1	0	2025-10-16 21:14:15
	mmartinez3	mmartinez3@empresa.com	1	0	2025-10-16 21:14:15
	csanchez4	csanchez4@hotmail.com	1	0	2025-10-16 21:14:15
	pgonzalez5	pgonzalez5@outlook.com	1	0	2025-10-16 21:14:15
	lgomez6	lgomez6@gmail.com	1	0	2025-10-16 21:14:15
	lfernandez7	lfernandez7@gmail.com	1	0	2025-10-16 21:14:15
	dsanchez8	dsanchez8@empresa.com	1	1	2025-10-16 21:14:15
	pperez9	pperez9@gmail.com	1	0	2025-10-16 21:14:15
	ssanchez10	ssanchez10@empresa.com	1	0	2025-10-16 21:14:15
	cgomez11	cgomez11@outlook.com	1	0	2025-10-16 21:14:15
	mrodriguez12	mrodriguez12@hotmail.com	1	0	2025-10-16 21:14:15

Explicación de su utilidad.

Simplifica la consulta de información relevante sobre los usuarios activos y sus credenciales. En lugar de tener que hacer un JOIN manual cada vez entre Usuario y CredencialAcceso, la vista ya lo deja listo. Podemos entonces:

- Monitorear usuarios activos: ver quiénes están habilitados (activo = TRUE).
- Revisar seguridad: comprobar cuándo fue el último cambio de contraseña (ultimoCambio).
- Detectar cuentas que deben actualizar su clave (requiereReset = TRUE).
- Generar reportes o estadísticas sobre usuarios activos sin tocar tablas base.

3.3 Evidencia de Interacción con IA

-Consultamos con la IA sobre que recomienda para generar la vista.

tengo que crear una vista útil, que me recomiendas?

¡Perfecto, Christian! Vamos a pensar juntos en una **vista útil** a partir de tus tablas `Usuario` y `CredencialAcceso`. Antes de proponer algo concreto, conviene reflexionar:

- 1. Objetivo de la vista**
 - ¿Qué información querés consultar más seguido?
 - Ejemplo: datos de usuarios activos y su última fecha de cambio de contraseña.
 - Otra opción: usuarios que necesitan reset de contraseña (`requiereReset = TRUE`).
- 2. Qué columnas incluir**
 - Normalmente conviene combinar datos de ambas tablas para no tener que hacer joins cada vez.
 - Podrías incluir: `username`, `email`, `activo`, `ultimoCambio`, `requiereReset`, `eliminado`.
- 3. Filtros y orden**
 - Una vista puede mostrar solo **usuarios activos**, o solo los que **requieren reset**, para facilitar reportes.
 - También podés ordenar por `fechaRegistro` ↓ `ultimoCambio` para ver los más recientes primero.
- 4. Ejemplo de idea conceptual de la vista (sin dar SQL directo aún)**
 - Nombre de la vista: `vw_UsuariosActivosConCredencial`
 - Filtra: `Usuario.activo = TRUE`
 - Muestra: `username`, `email`, `ultimoCambio`, `requiereReset`

Si querés, podemos **hacer juntos la sentencia SQL completa paso a paso**, asegurándonos de usar JOIN correctamente y aplicando los filtros que más te sirvan.

4 Etapa 4: Seguridad e Integridad

4.1 Creación de Usuario con Privilegios Limitados

Sentencias GRANT / REVOKE.

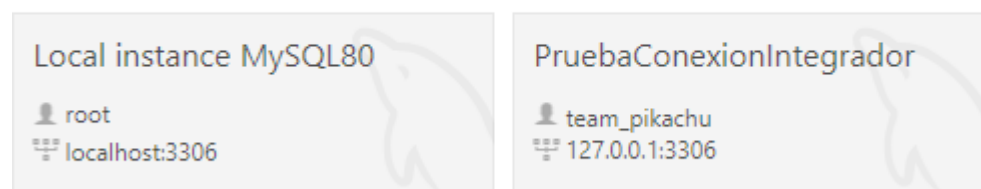
Prueba de acceso restringido.

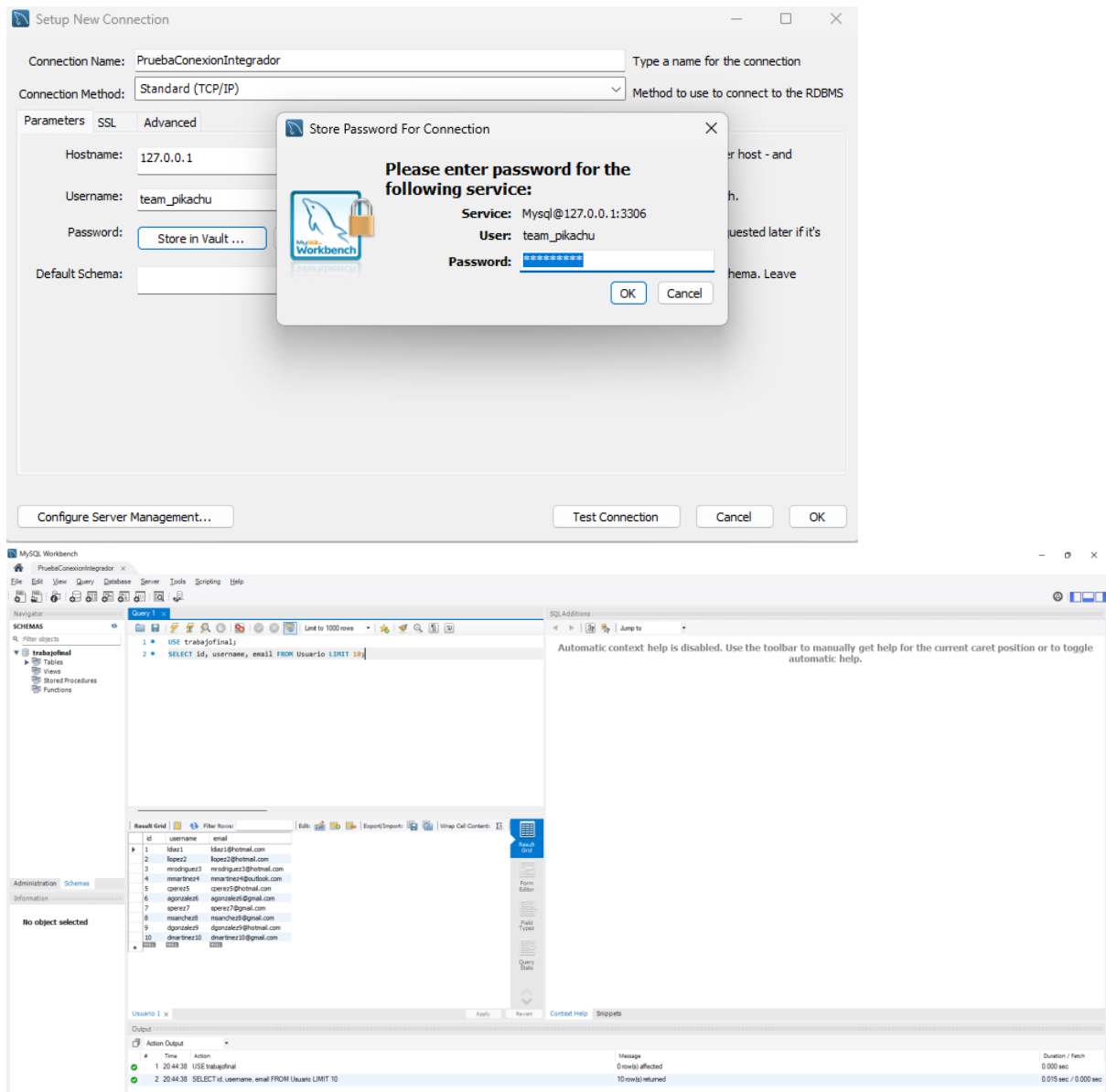
Para fortalecer la seguridad de la base de datos, se creó un nuevo usuario llamado `team_pikachu` con la sentencia `CREATE USER`, mediante la sentencia `GRANT`, se le asignaron privilegios de solo lectura sobre las tablas `Usuario` y `CredencialAcceso`, denegándole cualquier permiso de escritura o borrado. La validación de esta configuración se realizó simulando un acceso con las nuevas credenciales: se confirmó que las consultas de lectura funcionaban, mientras que un intento de borrado con `DELETE` fue bloqueado por el `MySQL`, mostrando el error 1142(`DELETE command denied`), demostrando la seguridad en la base de datos funciona correctamente. Si un futuro se desea eliminar el acceso a este usuario se utiliza la sentencia `REVOKE SELECT ON trabajofinal.CredencialAcceso FROM 'team_pikachu'@'localhost';`.

```
CREATE USER 'team_pikachu'@'localhost' IDENTIFIED BY  
'comision4';
```

```
GRANT SELECT ON trabajofinal.Usuario TO  
'team_pikachu'@'localhost';  
GRANT SELECT ON trabajofinal.CredencialAcceso TO  
'team_pikachu'@'localhost';
```

MySQL Connections





DELETE FROM Usuario WHERE id = 1;

20:45:45 DELETE FROM Usuario WHERE id = 1 Error Code: 1142. DELETE command denied to user 'team_pikachu'@'localhost' for table 'usuario' 0.015 sec

4.2 Vistas para Ocultar Datos Sensibles

```
USE trabajofinal;
-- vista 1
CREATE VIEW Vista_Usuarios_Publica AS
SELECT
    id,
    username,
    activo,
```

```

        fechaRegistro
FROM
    trabajofinal.Usuario;

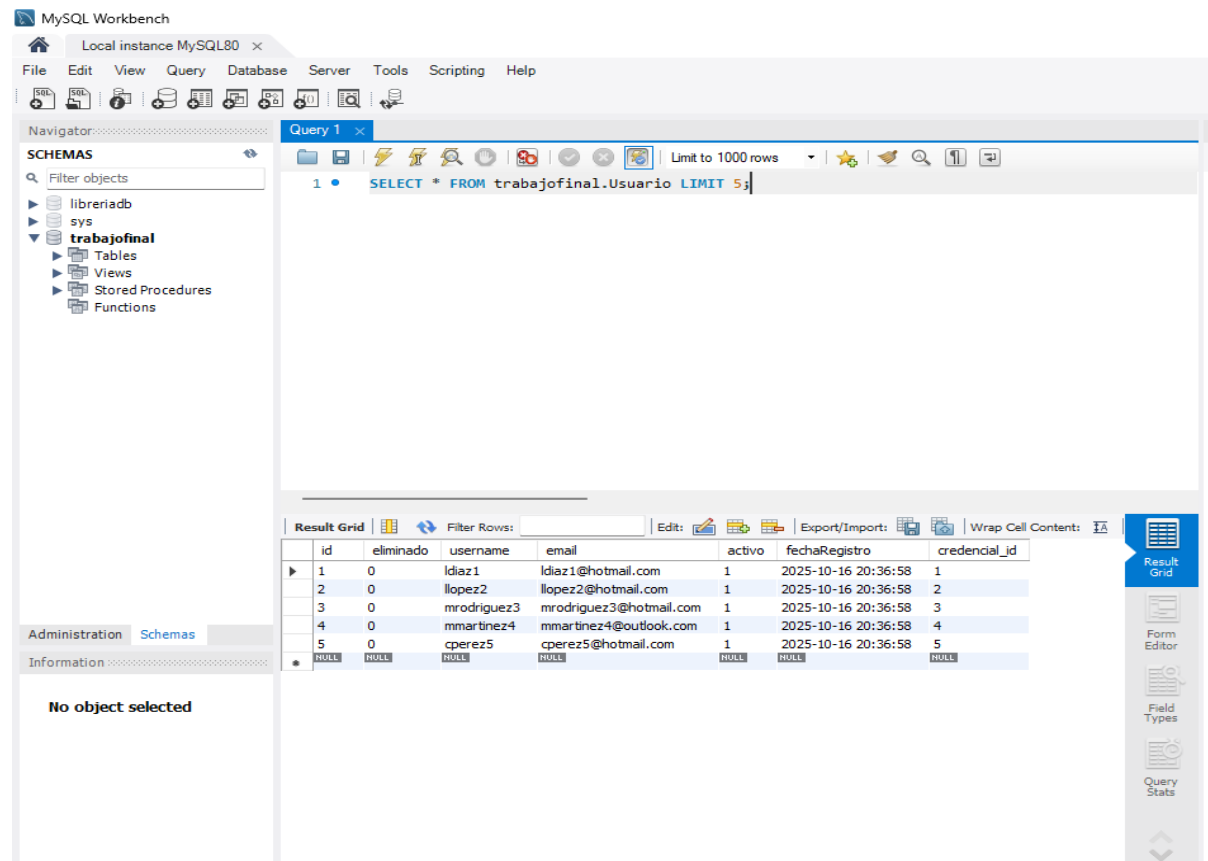
-- vista 2
CREATE VIEW Vista_Info_Seguridad AS
SELECT
    id,
    ultimoCambio,
    requiereReset
FROM
    trabajofinal.CredencialAcceso;

-- persimos para el usuario
GRANT SELECT ON trabajofinal.Vista_Usuarios_Publica TO
'team_pikachu'@'localhost';
GRANT SELECT ON trabajofinal.Vista_Info_Seguridad TO
'team_pikachu'@'localhost';
FLUSH PRIVILEGES;

```

4.2.1 Ejemplo de resultado (antes / después).

Antes:

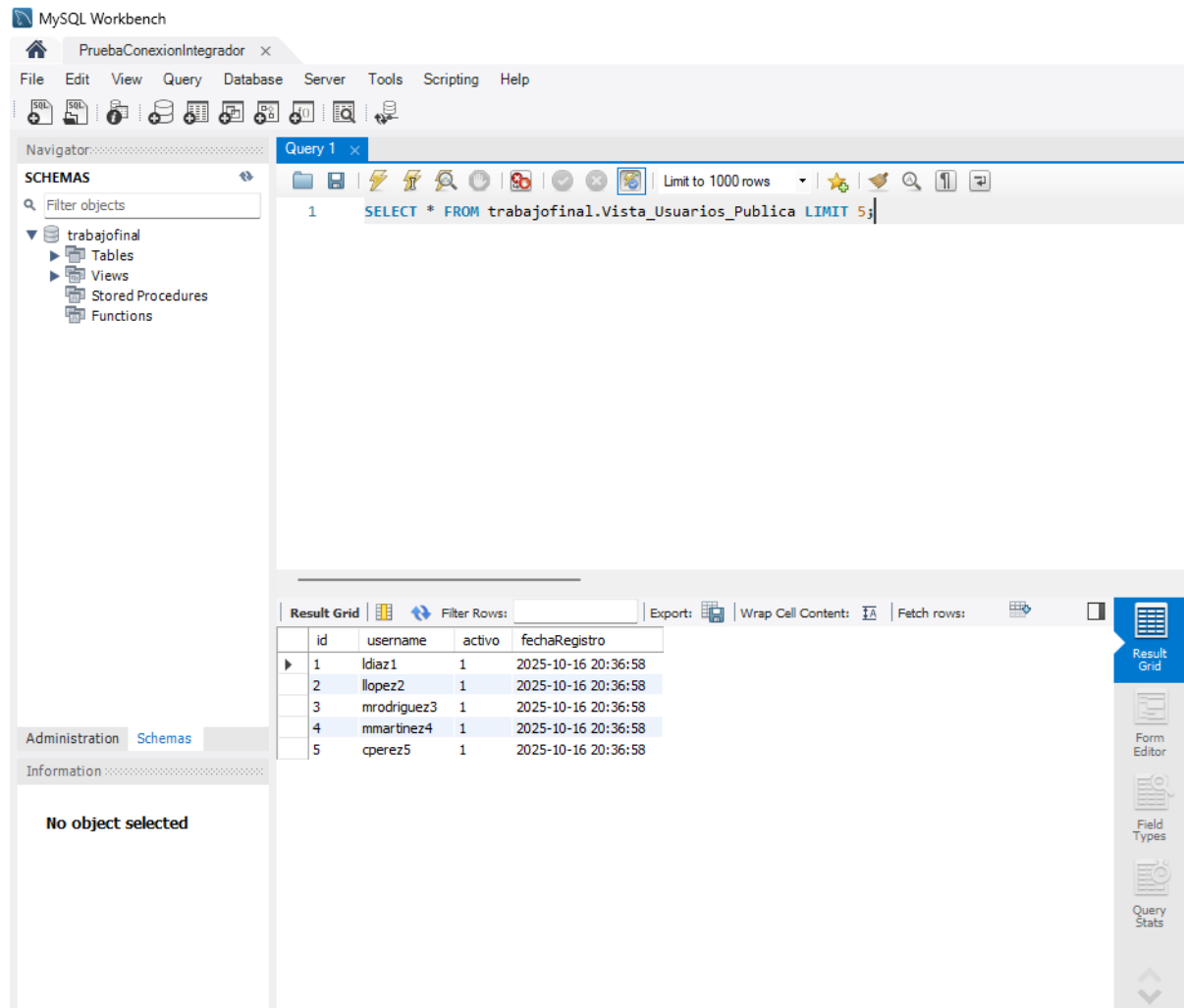


The screenshot shows the MySQL Workbench interface. The 'Navigator' pane on the left displays the database schema, including the 'trabajofinal' database. The 'Query 1' pane shows the SQL query: `SELECT * FROM trabajofinal.Usuario LIMIT 5;`. The 'Result Grid' pane displays the query results in a table format.

	id	eliminado	username	email	activo	fechaRegistro	credencial_id
1	0	ldiaz1	ldiaz1@hotmail.com	1	2025-10-16 20:36:58	1	
2	0	llopez2	llopez2@hotmail.com	1	2025-10-16 20:36:58	2	
3	0	mrodriguez3	mrodriguez3@hotmail.com	1	2025-10-16 20:36:58	3	
4	0	mmartinez4	mmartinez4@outlook.com	1	2025-10-16 20:36:58	4	
5	0	cperez5	cperez5@hotmail.com	1	2025-10-16 20:36:58	5	
	NULL	NULL	NULL	NULL	NULL	NULL	

The interface also shows a 'Schemas' pane on the left with a search filter, and a 'Result Grid' pane on the right with a 'Filter Rows' input field. The bottom status bar indicates 'No object selected'.

Después:



Para proteger la información sensible, creamos 2 vistas, que funcionan como filtros de seguridad para las tablas. Vista_Usuarios_Publica que muestra los perfiles de los usuarios pero oculta su email, y otra llamada Vista_Info_Seguridad que esconde los datos más críticos como las contraseñas cifradas.

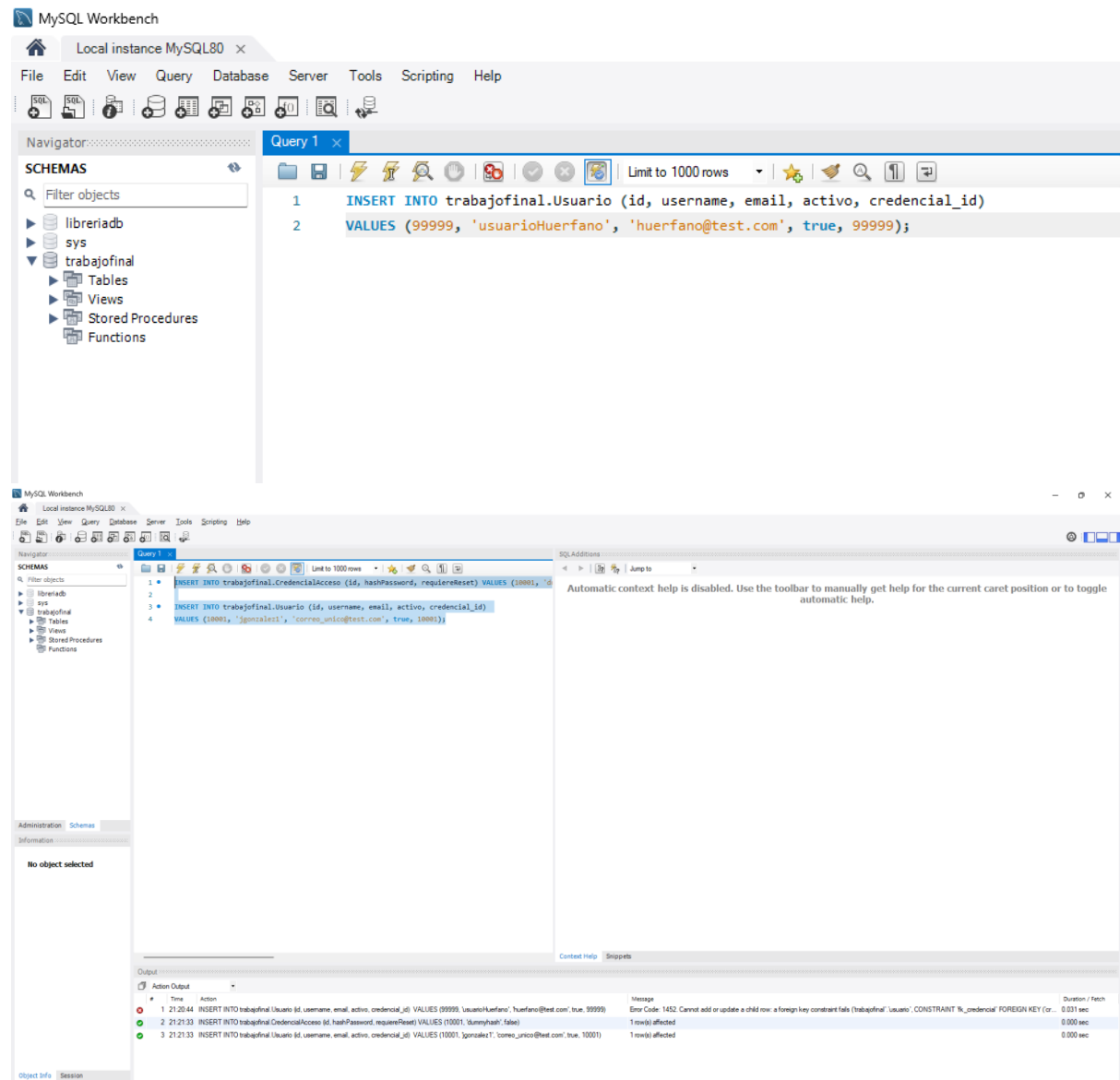
Para verificar si funciona el filtrado se realizó una consulta en ambos usuarios donde en admin muestra toda la información de la tabla y en el usuario team_pikachu solamente aparece el usuario, si está activo y la fecha de registro ocultando su correo.

4.3 Pruebas de Integridad

Se realizaron dos pruebas de integridad mediante inserciones fallidas para validar las restricciones de la base de datos. En la primera, se verificó la integridad referencial al intentar crear un usuario con un credencial_id inexistente; como se esperaba la base de datos rechazó la operación con el Error (1452), confirmando que la restricción de clave foránea funciona correctamente. La segunda prueba buscaba validar la restricción UNIQUE del username, pero un primer intento de insertar un nombre de usuario que se asumía duplicado fue exitoso de forma inesperada. Este resultado reveló que, debido a la naturaleza aleatoria del script de carga, el username elegido no existía realmente. Para validar la prueba de forma correcta, se

consultó un username existente y se reintentó la inserción, obteniendo en esta ocasión el Error Code: 1062 (Duplicate entry) esperado. Este proceso completo no solo demostró que ambas restricciones son robustas, sino que también subrayó la importancia de verificar el estado de los datos antes de realizar las pruebas.

Ejemplo 1 y 2 (con resultados).



4.3.1 Consulta Parametrizada Segura

Se eligió la implementación de una consulta segura mediante un procedimiento almacenado en MySql, ya que mantiene toda la lógica de seguridad dentro del propio motor de la base de datos.

Se creó un procedimiento llamado `BuscarUsuarioS` que acepta un nombre de usuario como parámetro de entrada y devuelve la información de dicho usuario.

```
USE trabajofinal;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE BuscarUsuarioS (IN p_username VARCHAR(30))
```

```
BEGIN
```

```
    SELECT
```

```
        id,
```

```
        username,
```

```
        email,
```

```
        activo,
```

```
        fechaRegistro
```

```
    FROM
```

```
        Usuario
```

```
    WHERE
```

```
        username = p_username;
```

```
END$$
```

```
DELIMITER ;
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'trabajofinal' selected. The main editor shows a query: `CALL BuscarUsuarioS('adiaz114');`. The bottom panel shows the 'Result Grid' with one row of data.

id	username	email	activo	fechaRegistro
114	adiaz114	adiaz114@hotmail.com	0	2025-10-16 20:36:59

The interface also includes a top menu bar, a toolbar, and a right sidebar with icons for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'.

4.4 Explicación del enfoque de seguridad.

El uso de procedimientos almacenados es una medida de seguridad eficaz contra la inyección SQL porque separa de forma estricta la estructura de la consulta de los datos de entrada. Al ejecutarse, el motor de la base de datos ya dispone de la estructura del SELECT pre compilada, por lo que cualquier valor recibido como parámetro se trata exclusivamente como un literal y no como parte del código ejecutable. Incluso si el parámetro contiene texto con sintaxis maliciosa, no puede modificar la lógica de la consulta ni ejecutar comandos adicionales, lo que protege la integridad de los datos y neutraliza el vector de ataque.

4.5 Evidencia de Interacción con IA

Uso de IA para buenas prácticas o revisión de código.

Se solicitó a Gemini, un mecanismo de seguridad para ocultar datos sensibles como email y contraseñas. La solución propuesta fue utilizar VIEW, que actúa como un filtro a nivel de columna. Gemini proporcionó el script completo para crear dos vistas, que oculta el email de los usuarios y otra que esconde el hashPassword.

La validación se demostró al mostrar el acceso de los distintos roles: mientras el usuario administrador mantiene la visibilidad total de los datos en las tablas, el usuario creado team_pikachu al realizar una consulta se confirmó que el email y contraseña estaban ocultos, probando que las vistas funcionan como una capa de seguridad efectiva para proteger los datos.

¡Claro que sí! Para ocultar datos sensibles como el email o las contraseñas, la mejor práctica en SQL es usar **Vistas**. Una vista funciona como una "tabla virtual" que muestra solo las columnas que tú elijas de una tabla real.

Aquí tienes el script completo para crear dos vistas de seguridad en tu base de datos y darle los permisos necesarios al usuario `team_pikachu`.

Script para Crear Vistas de Seguridad

SQL



```
-- 1. Seleccionamos la base de datos para asegurar que los comandos se apliquen en
USE trabajofinal;

-- 2. Creamos la primera vista para mostrar datos públicos del usuario.
-- Esta vista oculta intencionalmente la columna 'email', que es información privada.
CREATE VIEW Vista_Usuarios_Publica AS
SELECT
    id,
    username,
    activo,
    fechaRegistro
FROM
    Usuario;

-- 3. Creamos la segunda vista para información de seguridad no crítica.
-- Esta vista es muy importante porque oculta las columnas más sensibles: 'hashPasw' y 'password'.
CREATE VIEW Vista_Info_Seguridad AS
SELECT
    id,
    ultimoCambio,
```

Pregunta a Gemini



Herramientas

2.5 Pro



Gemini puede cometer errores, así que verifica sus respuestas.

5 Etapa 5: Concurrency y Transacciones

5.1 Simulación de Deadlock

Abrimos dos sesiones en Mysql Workbench en donde ambas están apuntando a la misma base de datos.

Ejecutamos la consulta 1, en la primera sesión e iniciamos la transacción en esa BD:

```
START TRANSACTION;  
UPDATE Usuario  
SET email = 'usuariol_mod@ejemplo.com'  
WHERE id = 1  
;
```

En esta consulta bloqueamos el usuario de la fila 1 con id = 1 (queda el bloque exclusivo en esa fila)

Ejecutamos la consulta 2 en la segunda sesión:

```
START TRANSACTION;  
UPDATE CredencialAcceso  
SET requiereReset = FALSE  
WHERE id = 1  
;
```

Bloqueamos credencialAcceso de la fila 1 con id= (queda el bloque exclusivo en esa fila, en la sesión 2)

Ejecutamos la consulta 3 en la primera sesión:

```
UPDATE CredencialAcceso  
SET requiereReset = TRUE  
WHERE id = 1  
;
```

Sesión 1 intenta modificar CredencialAcceso.id = 1, pero esa fila está bloqueada por la sesión 2

Ejecutamos la consulta 4 en la segunda sesión:

```
UPDATE Usuario  
SET email = 'bloqueado@ejemplo.com'  
WHERE id = 1  
;
```

Sesión 2 intenta modificar Usuario.id = 1, pero esa fila está bloqueada por la sesión 1

En ese punto, MySQL detecta un ciclo de espera mutua: cada transacción espera algo que la otra no puede liberar. Indicando en la última consulta el siguiente error:

Error Code: 1213. Deadlock found when trying to get lock; try restarting transaction

Después del error, consultamos qué pasó con el log interno de InnoDB por medio del comando:

```
SHOW ENGINE INNODB STATUS;
```

MySQL no deja colgadas dos sesiones, tiene un mecanismo interno que detecta el *deadlock* y elige una de las transacciones para abortar (rollback automático). Es decir, una transacción se “sacrifica” lanzando el error.

Como resumen del error y filtrando datos se pudo rescatar:

```
*** (1) TRANSACTION:
```

```
TRANSACTION 41337, ACTIVE 69 sec ...
```

```
...
```

```
*** (2) TRANSACTION:
```

```
TRANSACTION 41336, ACTIVE 75 sec ...
```

```
...
```

```
*** WE ROLL BACK TRANSACTION (1)
```

Había dos transacciones activas (la 41336 y la 41337). Ambas esperaban un recurso bloqueado por la otra. InnoDB eligió la transacción (1) como *víctima del deadlock* y la deshizo automáticamente (ROLL BACK TRANSACTION (1)).

MySQL detectó el ciclo de bloqueo y liberó los recursos deteniendo una sola transacción basándose en cuál transacción ha hecho menos trabajo (o lleva menos tiempo ejecutándose).

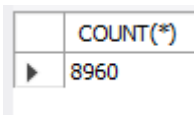
5.2 Manejo de Transacciones

Uso de BEGIN / COMMIT / ROLLBACK.

READ COMMITTED

Sesión 1, consulta 1:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT COUNT(*) FROM Usuario WHERE activo = TRUE
;
```



COUNT(*)
8960

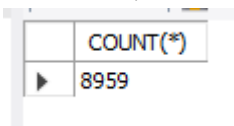
Aquí mantenemos la transacción abierta

Sesión 2, consulta 2:

```
UPDATE Usuario SET activo = FALSE WHERE id = 3;
COMMIT;
```

Sesión 1, Consulta 3:

```
SELECT COUNT(*) FROM Usuario WHERE activo = TRUE;
COMMIT;
```



COUNT(*)
8959

En el nivel de aislamiento READ COMMITTED cada SELECT lee solo los datos que han sido confirmados (COMMITTED) en ese momento.

No mantiene versiones "viejas" de las filas entre lecturas dentro de la misma transacción. Por eso, si otra transacción hace un UPDATE y COMMIT, la siguiente lectura ve el cambio.

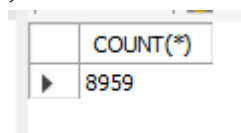
Durante la prueba, se ejecutaron lecturas en la sesión 1 y una actualización concurrente en la segunda sesión sin generar bloqueos ni errores.

Esto confirma que READ COMMITTED evita lecturas sucias (*dirty reads*), pero permite lecturas no repetibles, es decir, los resultados pueden cambiar entre dos lecturas dentro de la misma transacción.

REPEATABLE READ

Sesión 1, consulta 1:

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
START TRANSACTION;  
SELECT COUNT(*) FROM Usuario WHERE activo = TRUE  
;
```



	COUNT(*)
▶	8959

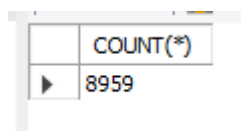
Aquí mantenemos la transacción abierta

Sesión 2, consulta 2:

```
UPDATE Usuario SET activo = TRUE WHERE id = 3;  
COMMIT;
```

Sesión 1, Consulta 3:

```
SELECT COUNT(*) FROM Usuario WHERE activo = TRUE;  
COMMIT;
```



	COUNT(*)
▶	8959

En REPEATABLE READ, los SELECT dentro de la misma transacción siempre ven los mismos datos, aunque otras sesiones hagan COMMIT.

Durante la prueba con REPEATABLE READ, la sesión 1 realizó dos lecturas dentro de la misma transacción. Aunque la sesión 2 actualizó y confirmó el cambio (COMMIT), la sesión 1 no vio ese cambio en su segunda consulta, ya que mantiene una vista consistente de los datos durante toda la transacción.

5.2.1 Procedimiento almacenado con BEGIN/COMMIT/ROLLBACK, manejo de errores y retry ante deadlock si aplica implementado en SQL.

Descripción del escenario: Inicia una transacción, actualizamos el Usuario y su CredencialAcceso. Si ocurre un deadlock o bloqueo, hace ROLLBACK, reintenta hasta 3 veces automáticamente. Si todo sale bien, hace COMMIT.

```
DELIMITER $$
CREATE PROCEDURE actualizarEmailConRetry(
    IN p_id BIGINT,
    IN p_email VARCHAR(120)
)
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE retries INT DEFAULT 0;
    DECLARE max_retries INT DEFAULT 3;

    -- Manejador de errores SQL (deadlock, bloqueo, etc.)
    DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    BEGIN
        SET done = 1;
        ROLLBACK;
    END;

    REPEAT
        SET done = 0;

        START TRANSACTION;
        BEGIN
            -- Actualiza el email del usuario
            UPDATE Usuario
            SET email = p_email
            WHERE id = p_id;

            -- También actualiza la fecha de cambio de
credencial
            UPDATE CredencialAcceso
            SET ultimoCambio = NOW()
            WHERE id = p_id;

            COMMIT;
        END;

        -- Si ocurrió error (deadlock o bloqueo)
```

```

        IF done = 1 THEN
            SET retries = retries + 1;
            IF retries < max_retries THEN
                -- Espera breve antes de reintentar
                DO SLEEP(0.5);
            END IF;
        END IF;

    UNTIL done = 0 OR retries >= max_retries
    END REPEAT;

END $$

DELIMITER ;
## Inicializar llamado
CALL actualizarEmailConRetry(1,
'nuevo\_correo@ejemplo.com');

```

El procedimiento actualizarEmailConRetry implementa una transacción que actualiza datos de dos tablas relacionadas (Usuario y CredencialAcceso).
Si se produce un bloqueo o deadlock, la operación se revierte con ROLLBACK y se reintenta hasta 3 veces.

SIMULACIÓN DE DEADLOCK: Se provocó un *deadlock intencional* ejecutando dos transacciones concurrentes que acceden a las tablas Usuario y CredencialAcceso en distinto orden.

Sesión 1

Consulta 1:

```

START TRANSACTION;
##bloqueamos primero la tabla Usuario
UPDATE Usuario
SET email = 'bloqueado_s12@ejemplo.com'
WHERE id = 1;

```

Sesion 2:

Consulta 2:

```

START TRANSACTION;
-- Bloquea primero la tabla CredencialAcceso
UPDATE CredencialAcceso
SET requiereReset = FALSE
WHERE id = 1;

```

Consulta 3:

```

-- Ahora intenta actualizar Usuario

```

```
UPDATE Usuario
SET email = 'bloqueado2@ejemplo.com'
WHERE id = 2;
```

Sesión 1:

Consulta 4:

```
UPDATE CredencialAcceso
SET requiereReset = FALSE
WHERE id = 2;
```

Entre las consultas 3 y 4 generamos el error de deadlock, Mysql elige la detener la consulta 3 mostrando el siguiente mensaje:

Error Code: 1213. Deadlock found when trying to get lock; try restarting transaction

Luego llamamos el procedimiento y resolvemos

```
CALL actualizarEmailConRetry(1, 'usuario_retry@ejemplo.com');
```

Si ocurre un error (1213), el procedimiento realiza ROLLBACK y reintenta la transacción hasta 3 veces.

Si no hay error, confirma los cambios con COMMIT.

El procedimiento actualizarEmailConRetry maneja automáticamente los errores de concurrencia, aplicando reintentos controlados ante *deadlocks* o bloqueos. Esto demuestra el uso de transacciones, control de errores y estrategias de recuperación en operaciones críticas concurrentes.

En la simulación se comprobó que el uso de transacciones y control de errores permite mantener la consistencia de los datos ante accesos simultáneos. El procedimiento implementado reduce el riesgo de fallos por *deadlocks* y garantiza la integridad de las actualizaciones.

5.3 Informe de Observaciones

Durante las pruebas de concurrencia se observó que MySQL maneja los bloqueos de fila automáticamente, pero pueden producirse *deadlocks* cuando dos transacciones acceden a las mismas filas en orden inverso. Se verificó que el nivel de aislamiento READ COMMITTED permite leer datos modificados por otras transacciones ya confirmadas, mientras que REPEATABLE READ mantiene la consistencia dentro de una misma transacción. Los índices ayudan a reducir la duración de los bloqueos al hacer más rápidas las búsquedas y actualizaciones. El manejo correcto de transacciones y el uso de ROLLBACK resultan esenciales para evitar inconsistencias en entornos concurrentes.

5.4 Evidencia de Interacción con IA

Cómo ayudó la IA a interpretar resultados o resolver el deadlock.

Lo que copiaste es el **reporte interno de InnoDB**, y confirma **exactamente lo que observaste en las sesiones**.

Vamos a leer y entender **las partes clave** del texto para que puedas usarlo en tu informe y explicar lo que pasó con fundamento técnico pero claro.

1 Identificación del *deadlock*

En el bloque del log aparece:

python-repl

 Copiar código

```
*** (1) TRANSACTION:
TRANSACTION 41337, ACTIVE 69 sec ...
...
*** (2) TRANSACTION:
TRANSACTION 41336, ACTIVE 75 sec ...
...
*** WE ROLL BACK TRANSACTION (1)
```



6 Anexo

El desarrollo del Trabajo Práctico Integrador abarcó cinco etapas, desde el modelado hasta la concurrencia. Durante la implementación y depuración, se utilizó la asistencia de IA (Gemini y ChatGPT) como apoyo técnico más el material de lectura de la materia lo cual se detalla a continuación:

Etapas 1 (Modelado): Se definieron y crearon las tablas Usuario y CredencialAcceso con sus restricciones. La IA asistió en la sintaxis SQL, como la equivalencia de tipos de datos (LocalDateTime a DATETIME) y la forma de forzar errores para las validaciones.

Etapas 2 (Carga Masiva): Se implementó un procedimiento (CargarUsuariosR) para generar 10.000 registros lo más realista posible. La asistencia de Gemini fue crucial debido que anteriormente se hizo un procedimiento más simple pero fuera de la realidad arrojando datos random sin sentido. Adicionalmente, proporcionó asistencia de depuración (debugging), diagnosticando la causa raíz de los errores críticos Error 2013 (timeout) y Error 1062 (duplicate entry). Para solucionar este último facilitó el script de limpieza (TRUNCATE TABLE) que fue esencial para restaurar el estado de las tablas y poder completar la carga masiva.

Etapas 3 (Consultas): Se desarrollaron consultas avanzadas, se creó un índice para mejorar la performance y se implementó una vista para reportes (v_usuarios_activos). La IA ayudó a conceptualizar el objetivo y la estructura de esta vista.

Etapas 4 (Seguridad): Se implementaron múltiples capas de seguridad. Gemini proporcionó los scripts para:

- Crear un usuario restringido (team_pikachu) y probar sus permisos.
- Crear vistas de seguridad (Vista_Usuarios_Publica, Vista_Info_Seguridad) para ocultar datos sensibles.
- Implementar un procedimiento seguro (BuscarUsuarioS) contra Inyección SQL. Además, ayudó a depurar una prueba de integridad (UNIQUE) que falló inesperadamente.

Etapas 5 (Concurrencia): Se simulaban deadlocks (Error 1213) y se probaron niveles de aislamiento (READ COMMITTED, REPEATABLE READ). La IA ayudó a interpretar los logs de INNODB STATUS para entender el deadlock y proporcionó la estructura de un procedimiento con lógica de reintento automático (actualizarEmailConRetry).